Machine Learning in Epidemiology with R – Part 2



Heart data and neural networks

Federica Gazzelloni

23 October 2026
Two-hour online, hands-on R workshop

01 Learning objectives and session route

Meet the heart data

Read the saved random forest results

Prepare inputs and build a network with Keras

Fit the network and evaluate it with mlr3

Compare mistakes and discuss the results

Explain the forest results, fit a neural network, and interpret its predictions.

02 West Nile surveillance and heart disease

Part 1	Part 2
West Nile surveillance	Clinical measurements
Define a prediction unit and outcome	One patient and existing disease
Preparation and evaluation	The same workflow, new learners

Use the same learning and evaluation ideas with a new dataset.

03

Meet the heart-disease data

270
patients

13
original predictors

2
outcome classes

heart_disease: 150 absent, 120 present

Examples: age, chest pain, cholesterol, exercise measurements

Checkpoint: What does one row represent? Which column is the outcome?

Saved random forest results

270 patients

180 training patients	90 test patients
The forest learned from these rows	These rows were kept aside
Training is already complete	Read the saved predictions today

Use the saved results today. We will not fit another random forest.

05

Correct predictions and mistakes

Observed outcome	Predicted present	Predicted absent
Present	30 true positives	7 false negatives
Absent	8 false positives	45 true negatives

False positive: predicts disease for a patient whose outcome is absent.

Discuss: what could a missed case or a false alarm mean for a patient?

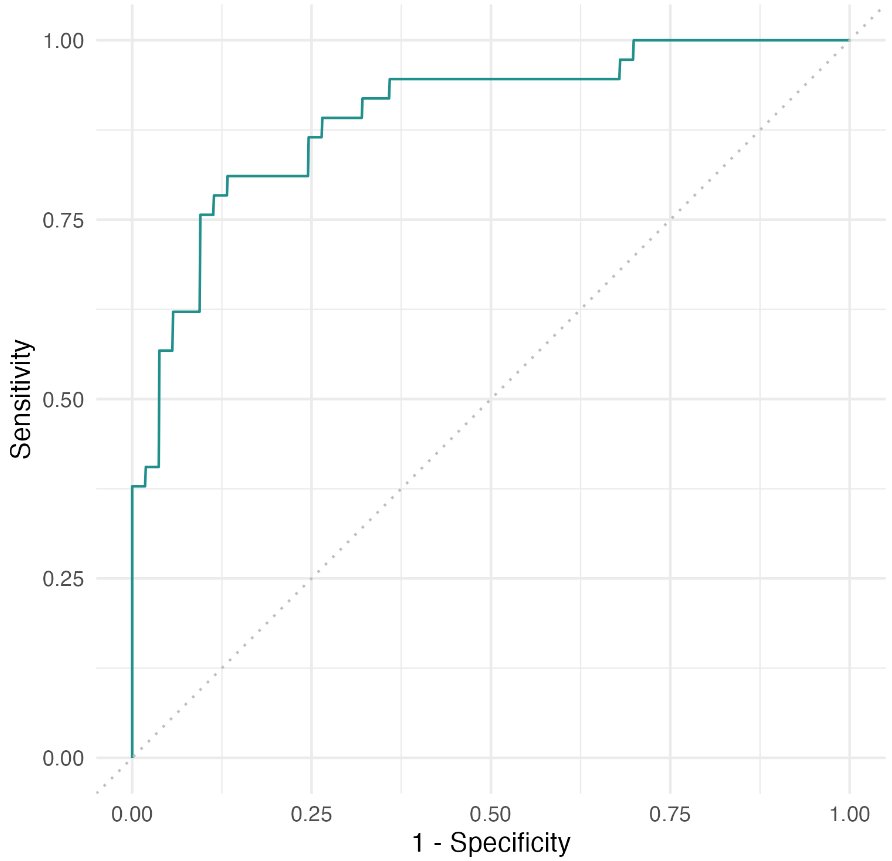
06

Accuracy, sensitivity and specificity

Measure	What does it count?	Saved result
Accuracy	Correct predictions: 75 / 90	83.3%
Sensitivity	Cases detected: 30 / 37	81.1%
Specificity	Non-cases identified: 45 / 53	84.9%

Accuracy alone can hide which patients the model gets wrong.

07 Probability thresholds and the ROC curve



Lowering the cut-off
usually catches more cases.

It also produces
more false alarms.

The curve compares choices
across different cut-offs.

Discuss: would accuracy alone tell us which cut-off to choose?

08 Organise the task with mlr3

Define the outcome, then reuse the saved training and test groups.

```
library(mlr3)
heart_task <- as_task_classif(
  heart,
  target = "heart_disease",
  positive = "present",
  id = "heart"
)
split_record <- read.csv("outputs/split.csv")
```

In notebook section 3, load the same 180 training and 90 test row numbers.

09 Prepare numerical inputs

Keep the outcome separate from the predictors.

Turn categories into 0/1 values and indicator columns.

Scale measurements using fitting patients only.

Check: ncol(x) is 22; y is 1 for present and 0 for absent.

Run the input-preparation block in notebook section 4.

A neural network learns weighted combinations

Inputs	Hidden layers	Output
Encoded measurements	Weighted combinations	Probability of present
Numbers the network receives	ReLU transforms learned signals	Sigmoid maps to 0–1

Training adjusts weights to reduce a loss: a measure of prediction error.

13 predictors become 22 encoded inputs

Input	Hidden layer 1	Hidden layer 2	Output
22 columns	64 units, ReLU	32 units, ReLU	1 unit, sigmoid
6 numeric + 3 binary + 13 indicators	Dropout 0.4	Dropout 0.4	P(disease present)

Dropout temporarily omits 40% of units during training.

The 13 measurements become 22 numerical input columns.

Build the network with Keras

Each layer passes its learned signals to the next layer.

```
library(keras3)
set_random_seed(20261023)

network <- keras_model_sequential(input_shape = c(22)) |>
  layer_dense(units = 64, activation = "relu") |>
  layer_dropout(rate = 0.4) |>
  layer_dense(units = 32, activation = "relu") |>
  layer_dropout(rate = 0.4) |>
  layer_dense(units = 1, activation = "sigmoid")
```

Run this definition after preparing x and y in the notebook.

Choose how the network learns

The optimiser updates weights; the loss measures prediction error.

```
compile(network,
  optimizer = optimizer_adam(learning_rate = 0.002),
  loss = "binary_crossentropy", metrics = "accuracy"
)
```

Binary cross-entropy measures error for this two-class outcome.

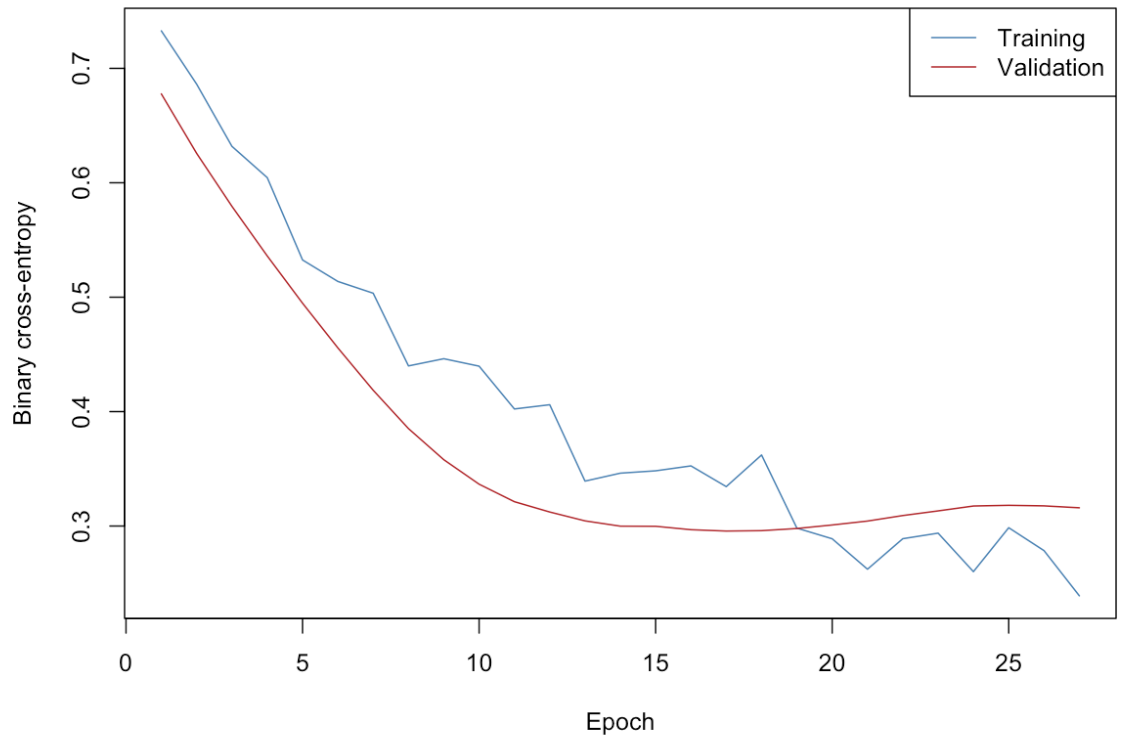
Fit the network

Keep 20% of the training group for validation while the network learns.

```
training <- fit(
  network,
  x[split$train, ],
  y[split$train],
  epochs = 50,
  batch_size = 32,
  validation_split = 0.2,
  verbose = 1,
  callbacks = list(callback_early_stopping(patience = 10, restore_best_weights = TRUE))
)
```

144 patients update weights; 36 monitor learning; 90 remain for final testing.

Read the training and validation loss



Training loss measures fit on rows used for learning.

Validation loss monitors rows kept out of updates.

Early stopping restores the best validation weights.

Run plot(training). The displayed figure is the saved workshop run.

Evaluate Keras predictions with mlr3

Predict the 90 test patients, then compare predictions with their outcomes.

```
prob <- as.numeric(predict(network, x[split$test, ], verbose = 0))
class <- ifelse(prob >= 0.5, "present", "absent")
network_pred <- PredictionClassif$new(
  task = heart_task, row_ids = split$test,
  truth = heart_task$truth(rows = split$test),
  response = class,
  prob = cbind(present = prob, absent = 1 - prob)
)
network_pred$confusion
network_pred$score(msrs(c("classif.acc", "classif.tpr", "classif.tnr")))
```

The same three measures let us compare the network with the saved forest.

Compare the saved model results

Measure	Random forest	Neural network
Accuracy	83.3%	85.6%
Sensitivity	81.1%	78.4%
Specificity	84.9%	90.6%

The saved network is more accurate overall, but misses one more case.

Workflow and limits

Read the saved forest results and explain its mistakes.

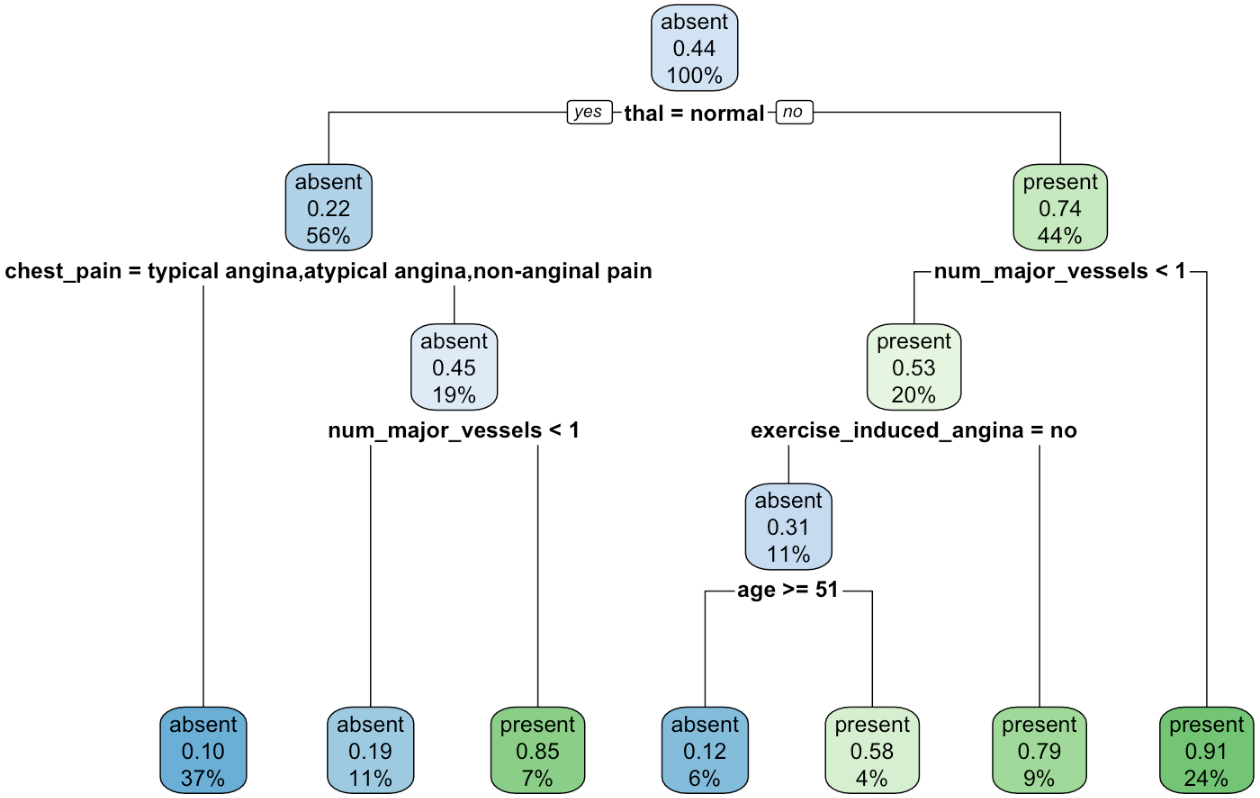
Prepare inputs, then build and fit the Keras network.

Use mlr3 to evaluate predictions on the same test patients.

Compare accuracy and error types before drawing conclusions.

These teaching results establish neither clinical validation nor causation.

Appendix: how a decision tree works



A split asks a question about one predictor.

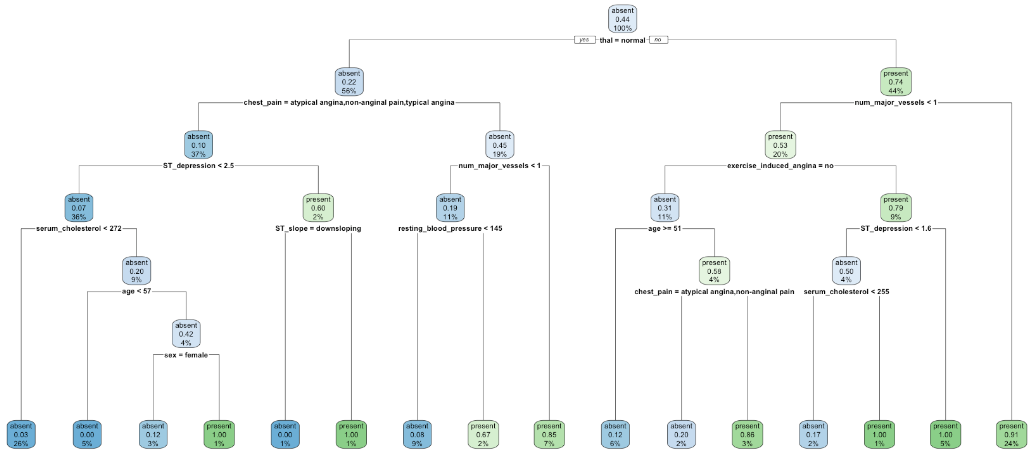
Follow a branch until you reach a leaf.

A leaf gives a class and fitted proportions.

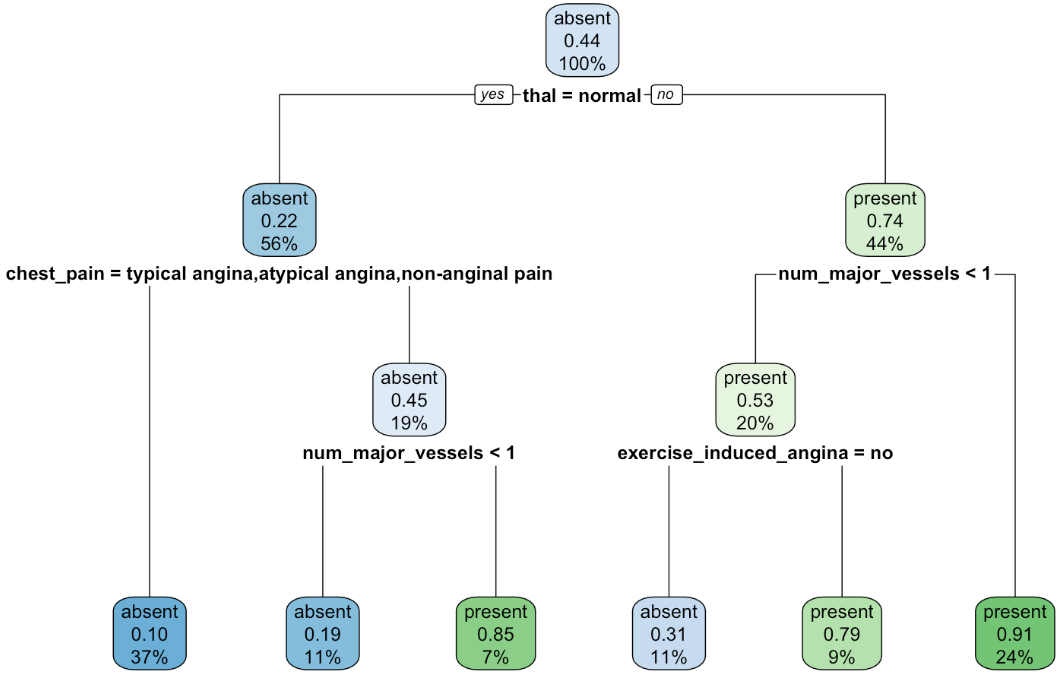
Default tree: fitted on all 270 patients to illustrate learning.

Appendix: deep and pruned trees

Deep tree: minsplit = 2

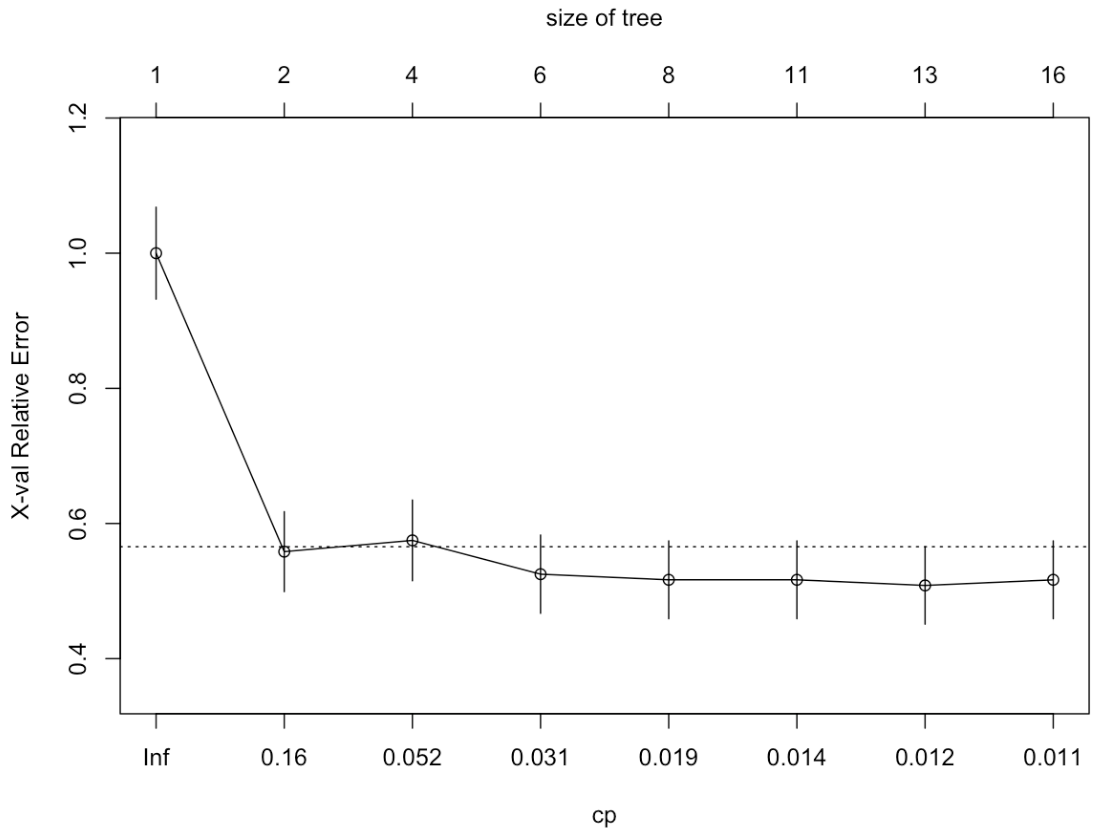


Pruned tree: cp = 0.031



Compare both with the default tree. More branches can fit details of this sample.

Appendix: overfitting and pruning



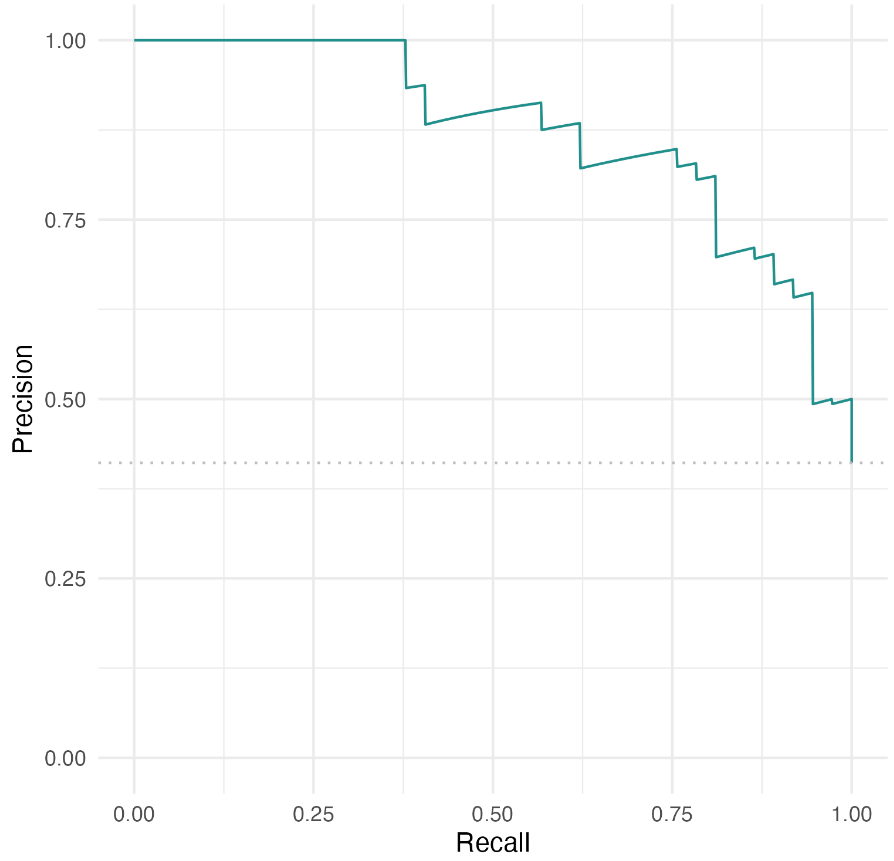
Overfitting learns sample details that may not repeat.

Pruning removes branches to limit complexity.

cp = 0.031 retains the article's replication setting.

Checkpoint: Why might pruning help predictions on new patients?

Appendix: precision and recall



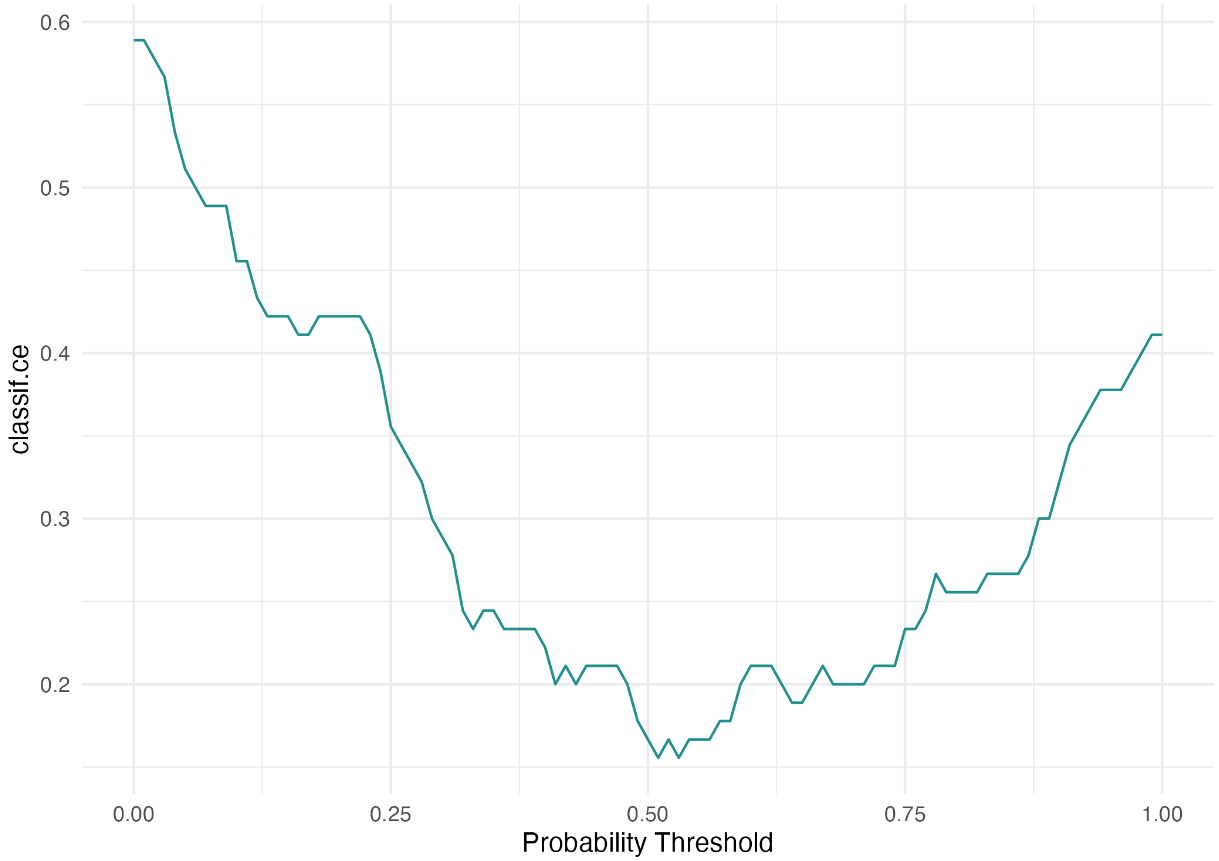
Recall = sensitivity.

Precision = positive predictive value.

The curve shows their tradeoff across thresholds.

Disease frequency matters when interpreting precision and comparing datasets.

Appendix: probability thresholds



Lower threshold usually raises sensitivity and reduces specificity.

Higher threshold usually reduces sensitivity and raises specificity.

Checkpoint: How would lowering the threshold affect missed disease and false alarms?

Appendix: resampling

Cross-validation rotates which fold stays out of fitting.

Repeated subsampling draws different training/test splits.

Bootstrap samples training rows with replacement.

Completed locally: 3-fold CV, 100 subsamples and 40 bootstrap repetitions.

Appendix: tuning and nested cross-validation

A hyperparameter controls how a model learns.

Inner resampling selects settings within training data.

Outer folds assess the whole selection procedure.

Full grid search and full-budget nested CV remain unexecuted. Only reduced smoke runs passed.

Appendix: interpretation

Approach	Question
Feature importance	Which predictors contribute to this fitted model?
Feature effects	How do predictions vary with a predictor?
Local explanation	What helps explain this one prediction?

Conceptual extension. No computed feature ranking or local explanation is supplied.

Appendix: synthetic observations

A generator learns patterns from the observed data.

Generate artificial records with the fitted distribution.

Compare age summaries and outcome proportions with heart.

Generated rows are synthetic examples. They are not additional observed patients.

Appendix: sources and reproducibility

Wright et al. (2026), Machine Learning in Epidemiology
arXiv:2602.16352v1, sections 2–6

OpenML heart-statlog, dataset 53, version 1

Local notebook, scripts, saved figures and output records

Source details, timing and evidence gaps accompany this deck.